

RUHR-UNIVERSITÄT BOCHUM

Fundamentals of GPU Programming, Lecture 3

Denis Eremin

Fakultät für Elektrotechnik und Informationstechnik
Lehrstuhl für Theoretische Elektrotechnik

CUDA Execution Model

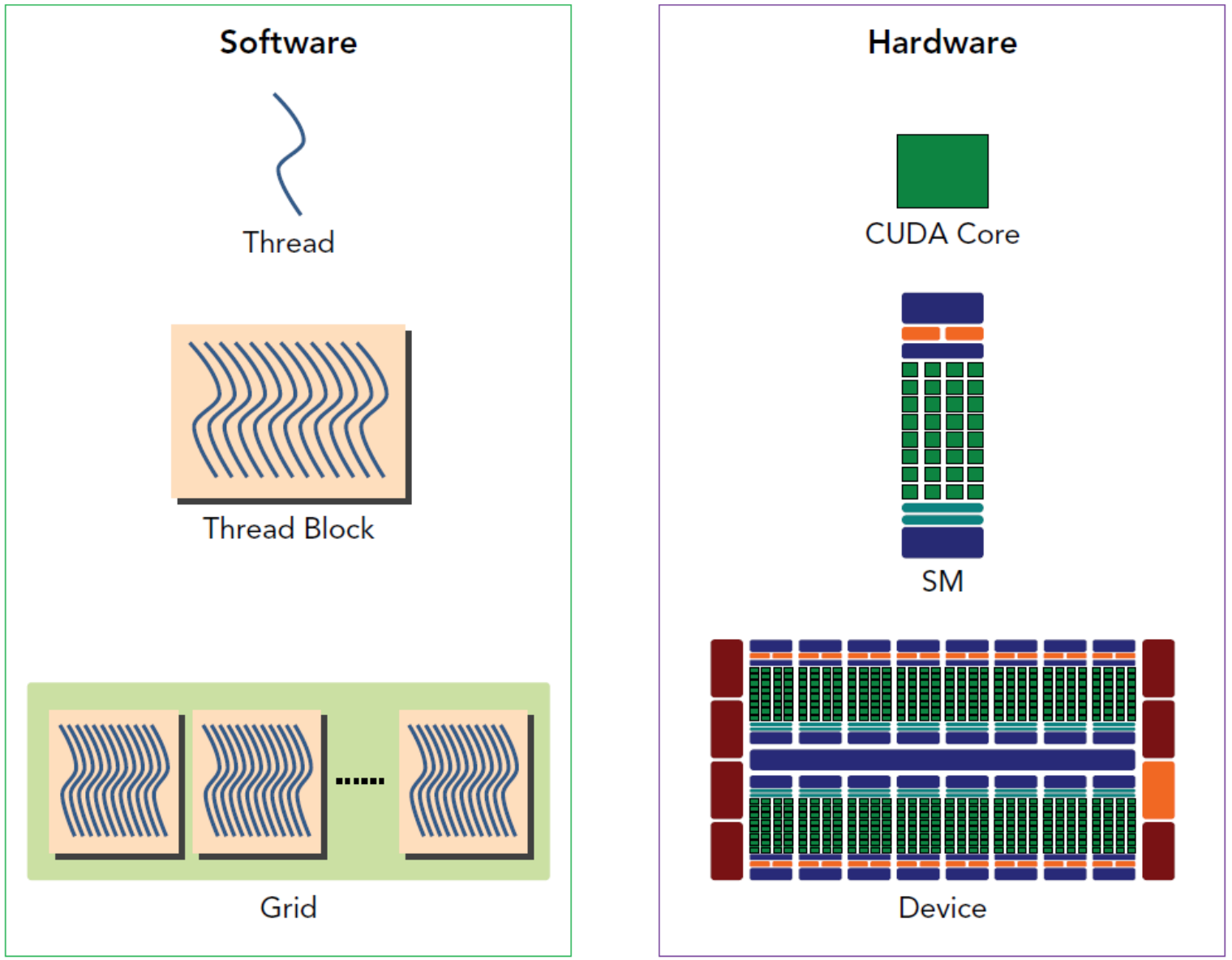
We will learn

- Thread scheduling, warps, latency hiding
- Occupancy
- Thread Level Parallelism vs Instruction Level Parallelism

Example: Matrix Addition

KERNEL CONFIGURATION	KERNEL ELAPSED TIME	BLOCK NUMBER
(32,32)	0.060323 sec	512 x 512
(32,16)	0.038041 sec	512 x 1024
(16,16)	0.045535 sec	1024 x 1024

Cuda Execution Model



Main task of a skillful thread scheduling (by shaping your execution configuration) : max hardware utilization subject to hardware limitations!

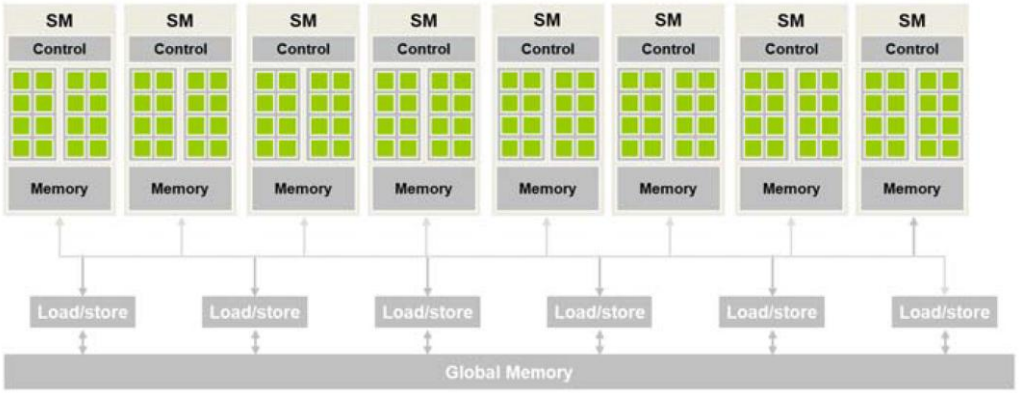
Depending on a problem, max hardware utilization could mean different things (maximization of the thread level parallelism, TLP /instruction level parallelism, ILP /task parallelism, TP), but they all should lead to maximization of the code performance!

Block Granularity, Hardware Limits

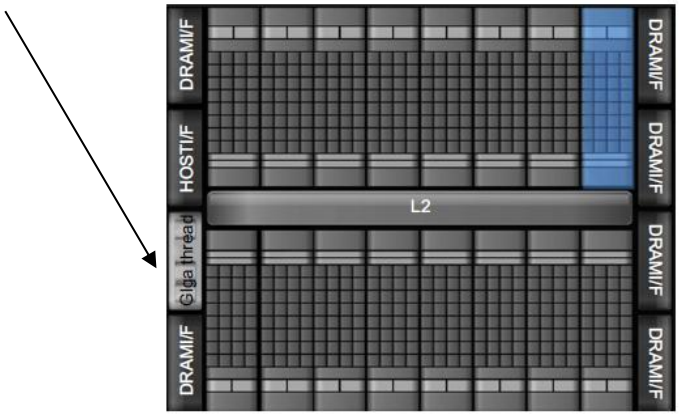
Two different types of hardware limitations

- Limits on the maximal number of resident threads and blocks on a SM, respectively (fixed number of thread and block slots per SM) caused by the limited number of hardware counters on GPU required for the warp management
- Limits on the max size of used memory resources: register/shared memory

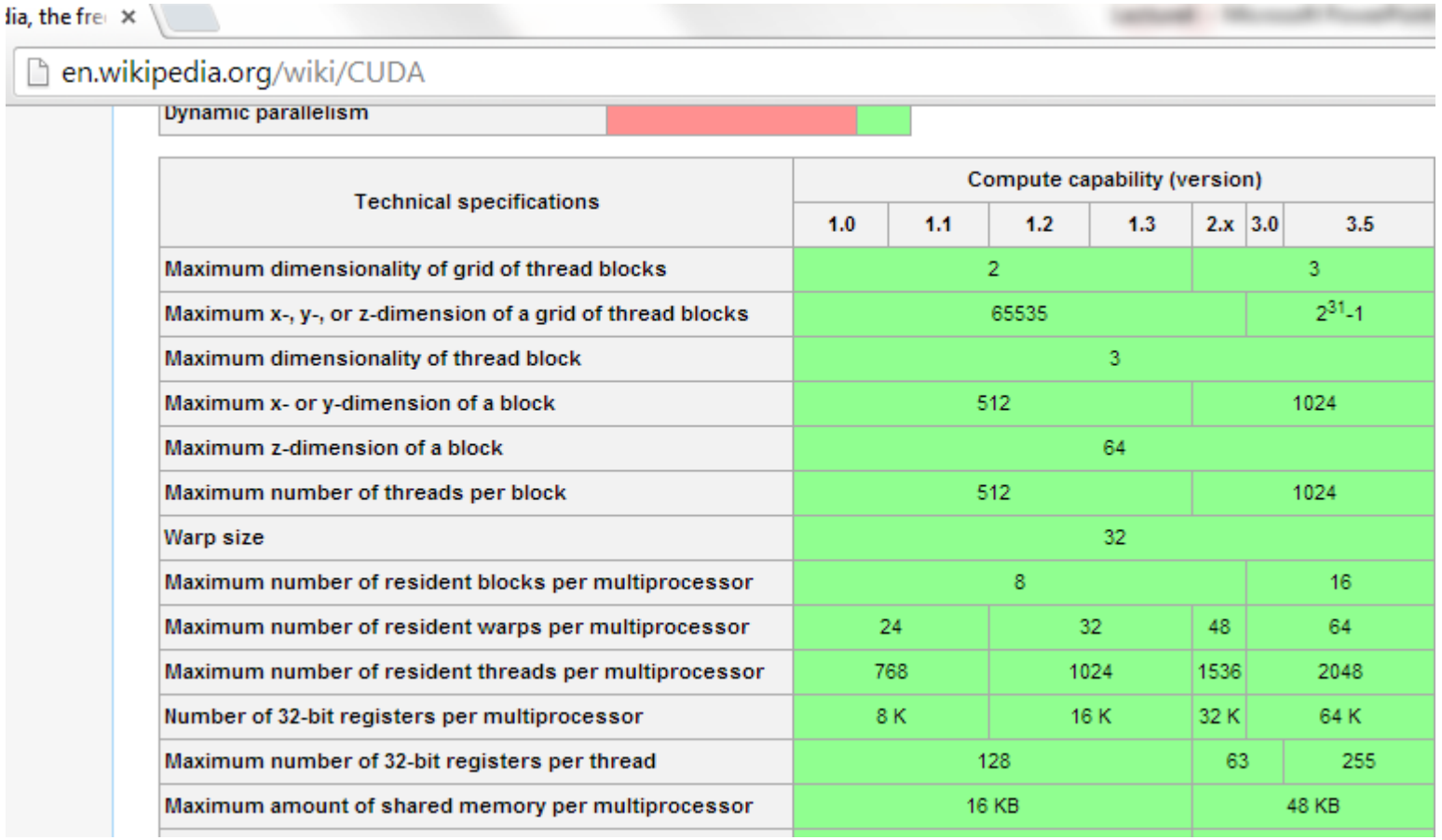
GPU architecture



GigaThread Engine
(block scheduler)



Technical Block Size Limit (Wikipedia)

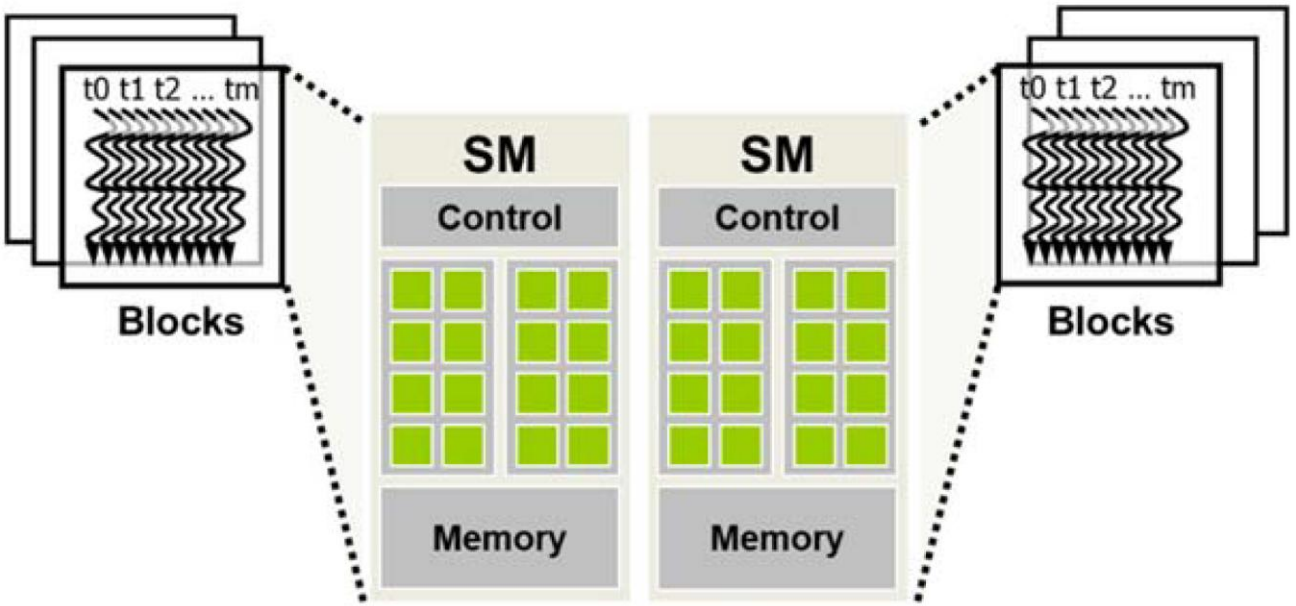


lia, the fre ×

en.wikipedia.org/wiki/CUDA

Dynamic parallelism

Technical specifications	Compute capability (version)						
	1.0	1.1	1.2	1.3	2.x	3.0	3.5
Maximum dimensionality of grid of thread blocks	2				3		
Maximum x-, y-, or z-dimension of a grid of thread blocks	65535						2^{31}-1
Maximum dimensionality of thread block	3						
Maximum x- or y-dimension of a block	512				1024		
Maximum z-dimension of a block	64						
Maximum number of threads per block	512				1024		
Warp size	32						
Maximum number of resident blocks per multiprocessor	8						16
Maximum number of resident warps per multiprocessor	24		32		48		64
Maximum number of resident threads per multiprocessor	768		1024		1536		2048
Number of 32-bit registers per multiprocessor	8 K		16 K		32 K		64 K
Maximum number of 32-bit registers per thread	128				63		255
Maximum amount of shared memory per multiprocessor	16 KB				48 KB		



- Threads are assigned to Streaming Multiprocessors in block granularity
  - Up to **8** blocks to each SM as resource allows
  - SM in GF100 can take up to **1536** threads
    - Could be 512 (threads/block) * 3 blocks
    - Or 256 (threads/block) * 6 blocks
- Threads run concurrently
  - SM maintains thread/block id #s
  - SM manages/schedules thread execution

Example: GF100 Block Granularity Considerations

- For Matrix Multiplication using multiple blocks, should I use 8X8, 16X16 or 64X64 blocks?
  - For 8X8, we have 64 threads per Block. Since each SM can take up to 1536 threads, there are 24 Blocks. However, each SM can only take up to 8 Blocks, only 512 threads will go into each SM!
  - For 16X16, we have 256 threads per Block. Since each SM can take up to 1536 threads, it can take up to 6 Blocks and achieve full capacity unless other resource considerations overrule.
  - For 64X64, we have 4096 threads per Block. Not even one can fit into an SM!

Other HW Limiting Factors: Memory Size

- The number of registers per SM (in Fermi) is 32768 bytes;

  If 1536 threads, approx. 21 32-bit registers per thread

- The amount of shared memory available per SM (in Fermi) is 49152 bytes;

  If 8 blocks, approx. 6 Kb per block

- The number of blocks and warps that can reside and be processed on SM depends on
  1) max number of resident blocks and warps per SM
  2) register and shared memory used

- If there are no sufficient registers or shares memory on SM per single block, the kernel will fail to launch

  Example:

  Q: considering the matrix multiplication algorithm with 16 x 16 blocks
  and max size of register memory on a SM equal to 16,384 floats,
  how many threads can be put on the SM if the kernel uses 10 and 12 registers
  per thread, respectively?

  A: For 10 registers per thread total number of registers in a block is 2,560, and for 6 blocks it is 15,360. For 12 registers per thread total number of registers in a block is 3,072 and for 6 blocks it is 18,432.

Obtaining Device Properties

Obtaining device properties in your code allows to tailor the kernel execution configurations for maximizing the kernel performance!

```
int dev_count;
cudaGetDeviceCount( &dev_count);
```

Number of GPUs

```
  cudaDeviceProp dev_prop;
  for (I = 0; i < dev_count; i++) {
    cudaGetDeviceProperties( &dev_prop, i);
    // decide if device has sufficient resources and capabilities
}
```

Device Properties

dev_prop is a C structure with different fields giving all the necessary information about a particular GPU (GPUs) in the system

Thread Assignment

Compute capability 2.0 (e.g., Tesla C2050)

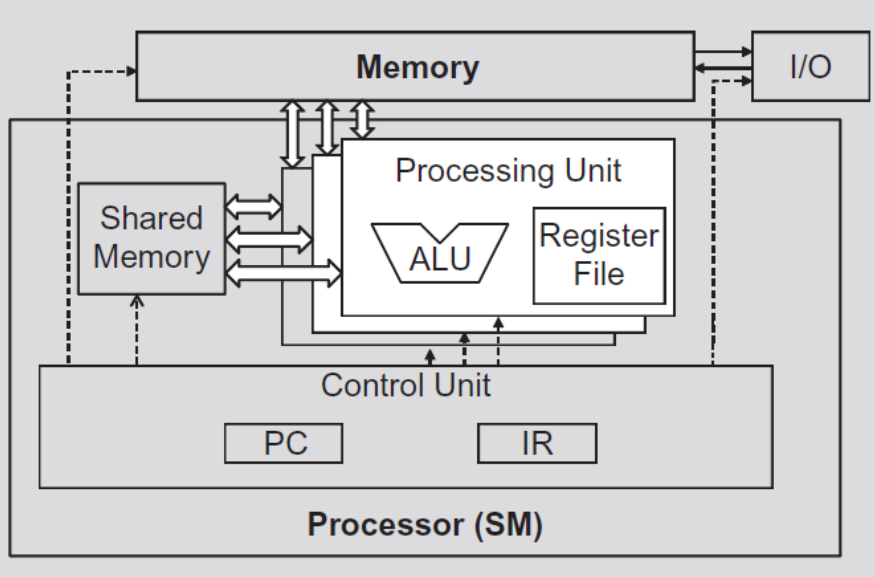
Device 0: “Tesla C2050"
CUDA Capabitily Major/Minor revision number: 2.0
Total amount of global memory: 2817982464 bytes
Total amount of shared memory per block: 49152 bytes
Total number of registers available per block: 32768 bytes
Warp size: 32
Maximum number of threads per block: 1024
Maximum sizes of each dimension of a block: 1024 x 1024 x 64
Maximum sizes of each dimension of a grid: 65535 x 65535 x 65535

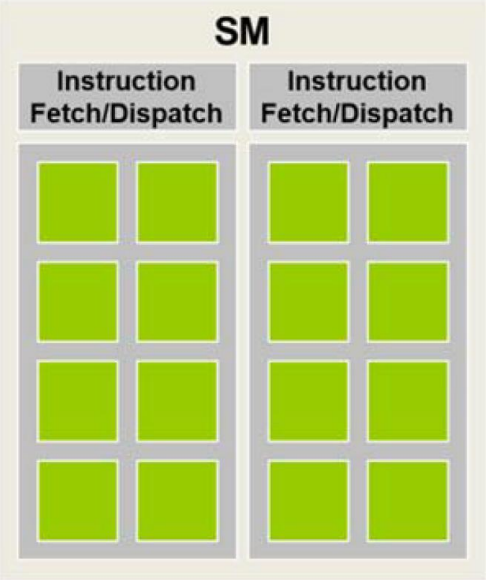
- - The maximum number of resident blocks per SM is 8
- - The maximum number of active threads per SM is 1536
- - The size of register memory per SM is 32 kB
- - The maximum size of shared memory is 48 kB

Thread Scheduling Units: Warps

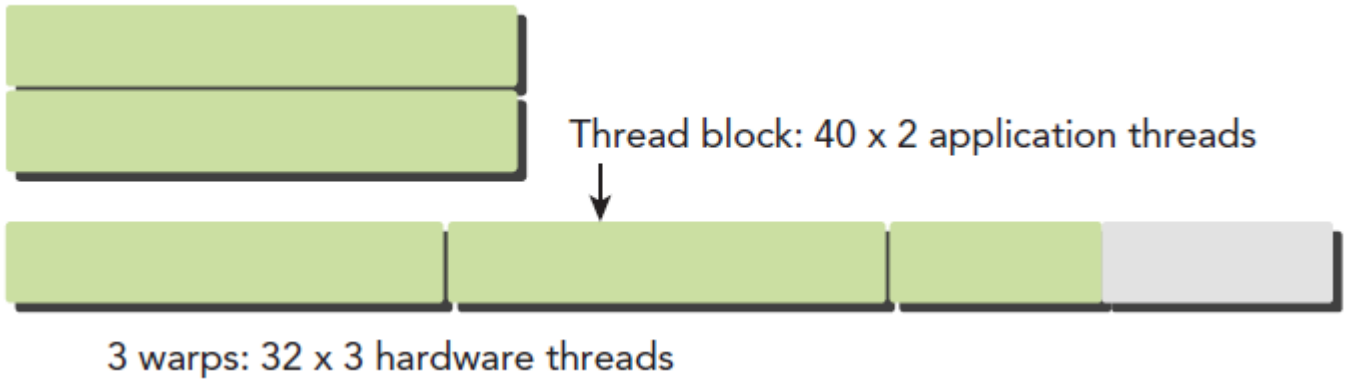
- SM creates, manages, schedules and executes threads in groups of 32 parallel threads called warps (threads in warps are free for branching)
- SM partitions blocks into warps to execute and schedule
- Fully efficient execution when no branching; if branching, each path is executed serially, then converge to the same execution path

SIMD



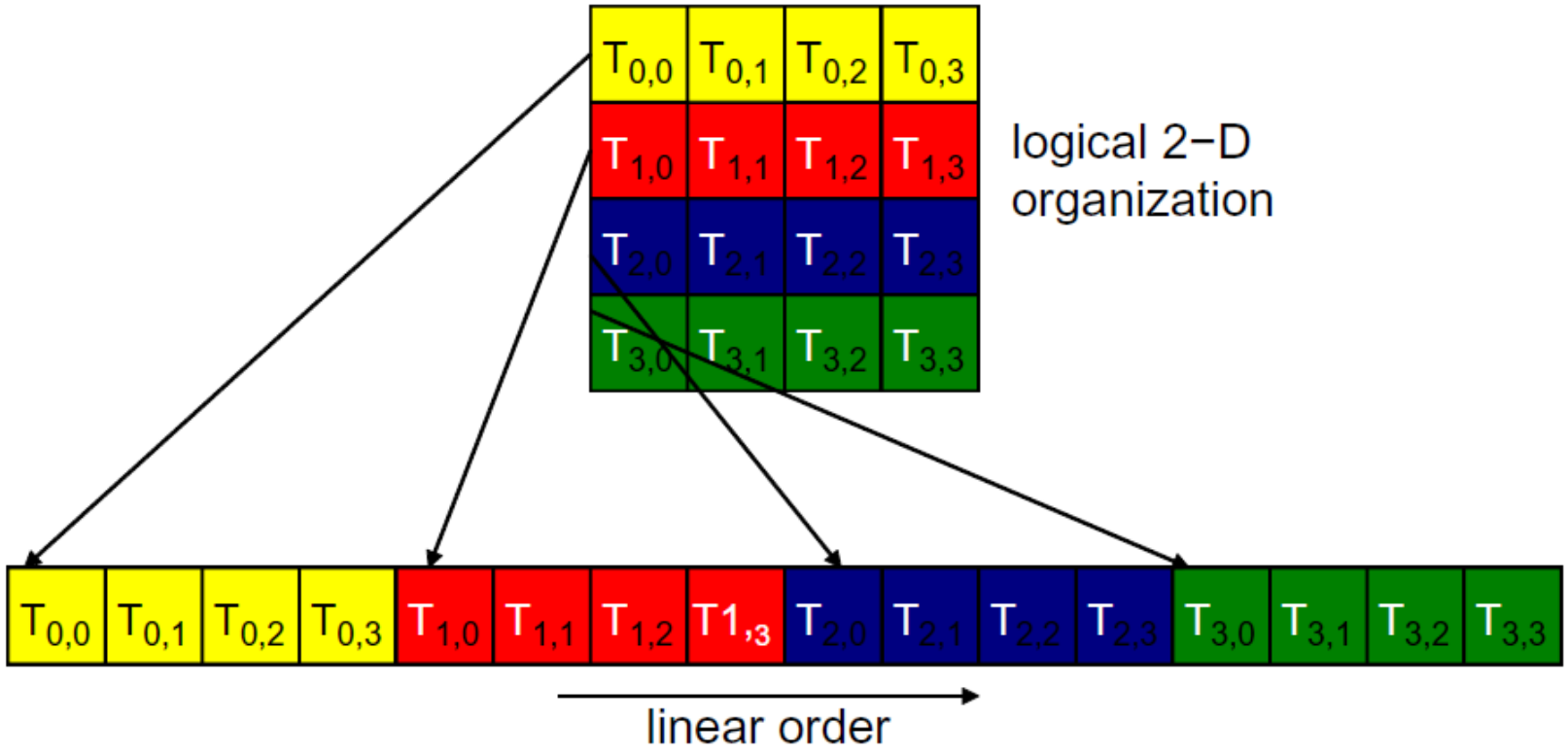


$$WarpsPerBlock = ceil\left(\frac{ThreadsPerBlock}{warpSize}\right)$$



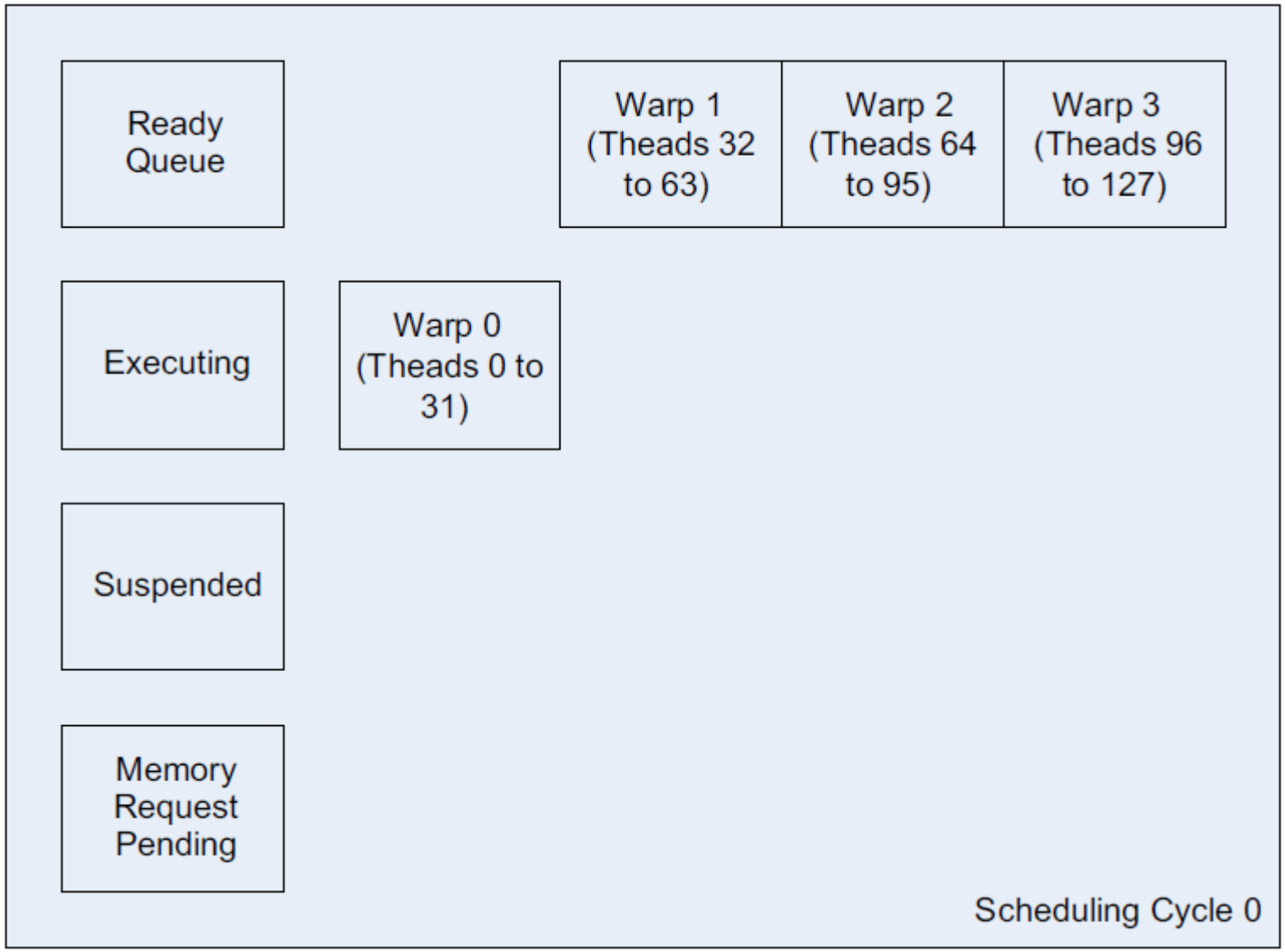
Thread Scheduling Units: Warps

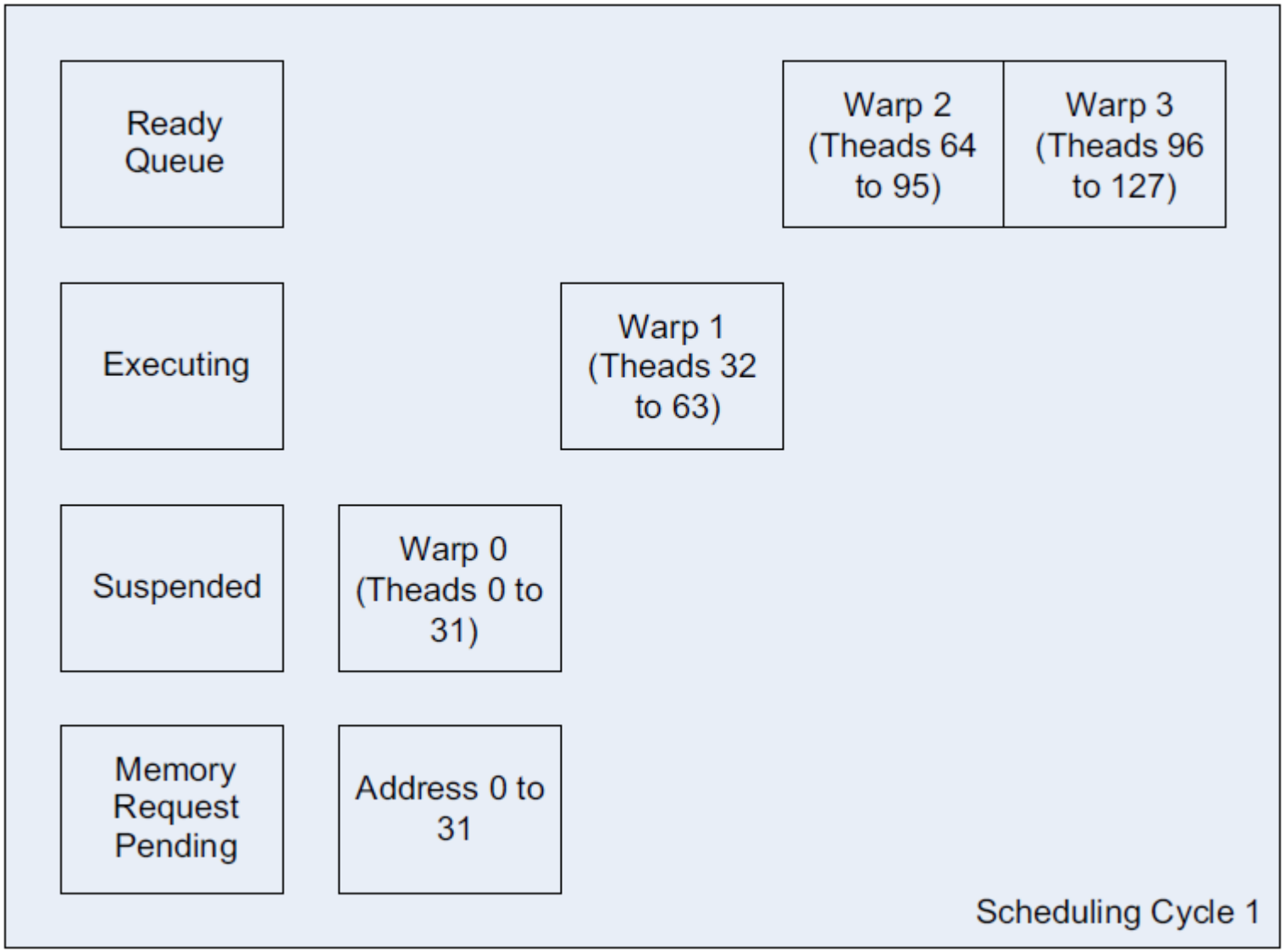
Threads in a warp have consecutive (linear) Id!

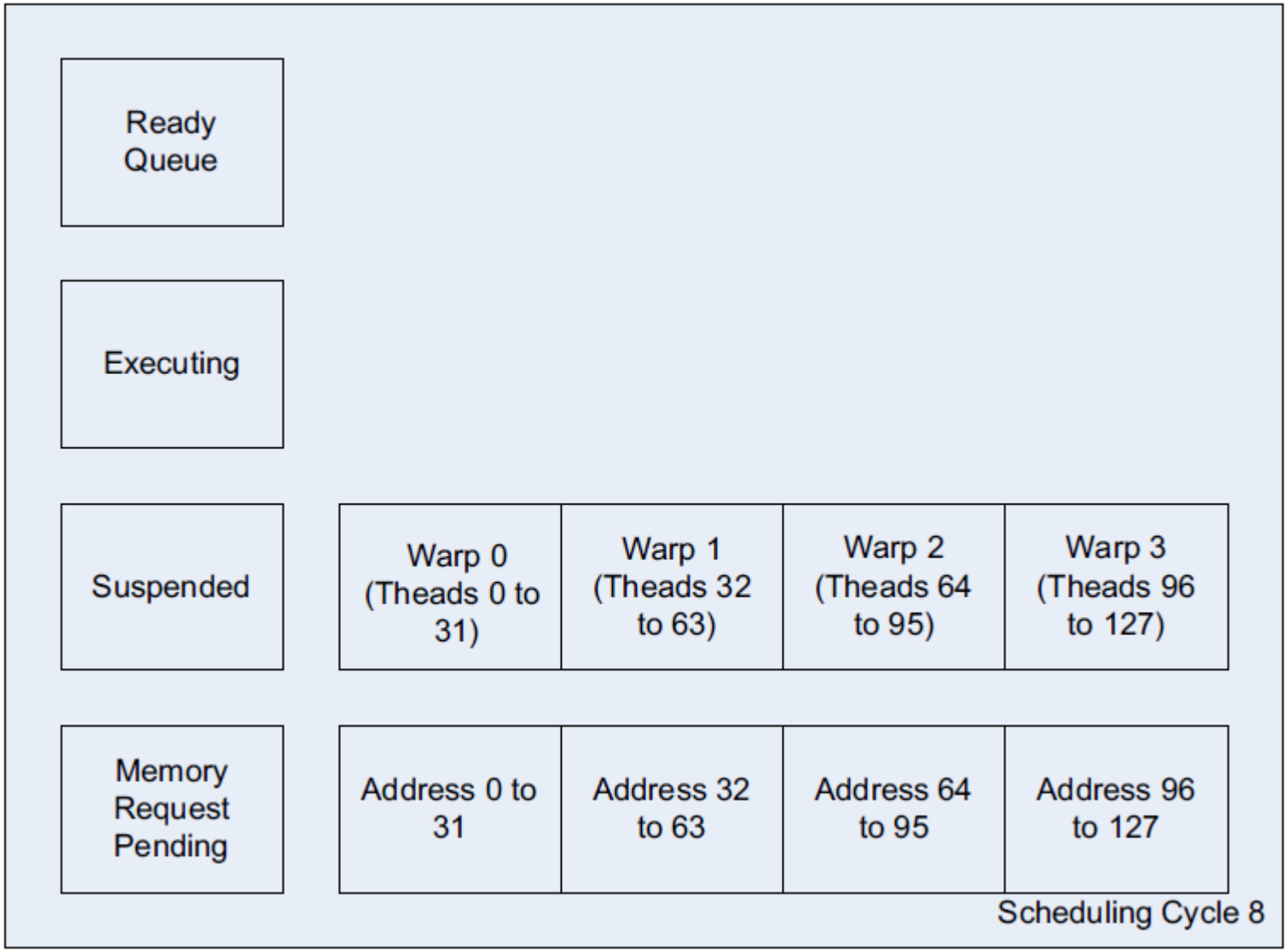


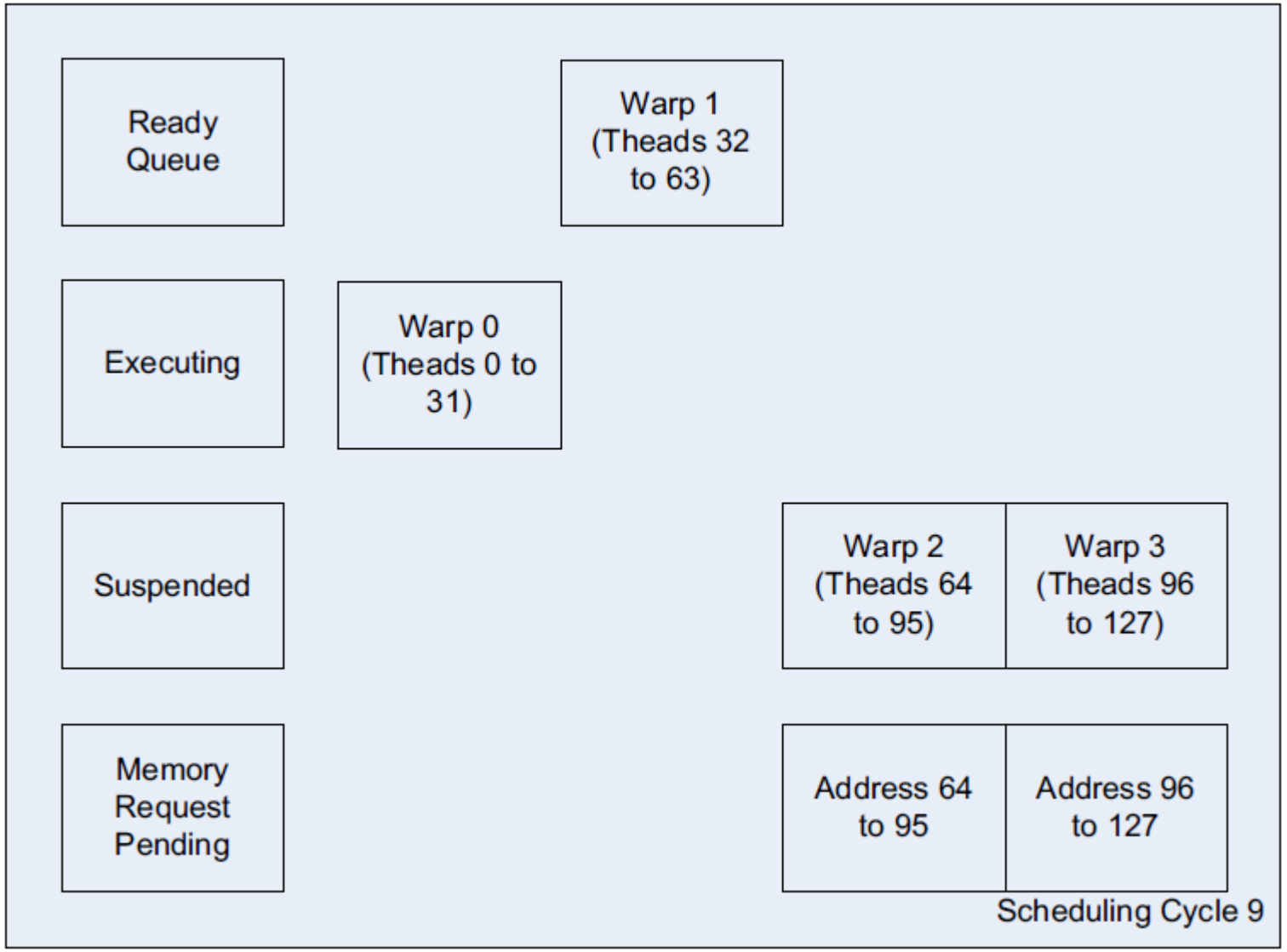
Active (resident) Warps:

- selected
- stalled
- eligible


Ready Queue
Warp 1 (Theads 32 to 63)
Warp 2 (Theads 64 to 95)
Warp 3 (Theads 96 to 127)
Executing
Warp 0 (Theads 0 to 31)
Suspended
Memory Request Pending
Scheduling Cycle 0


Ready Queue
Warp 2 (Theads 64 to 95)
Warp 3 (Theads 96 to 127)
Executing
Warp 1 (Theads 32 to 63)
Suspended
Warp 0 (Theads 0 to 31)
Memory Request Pending
Address 0 to 31
Scheduling Cycle 1

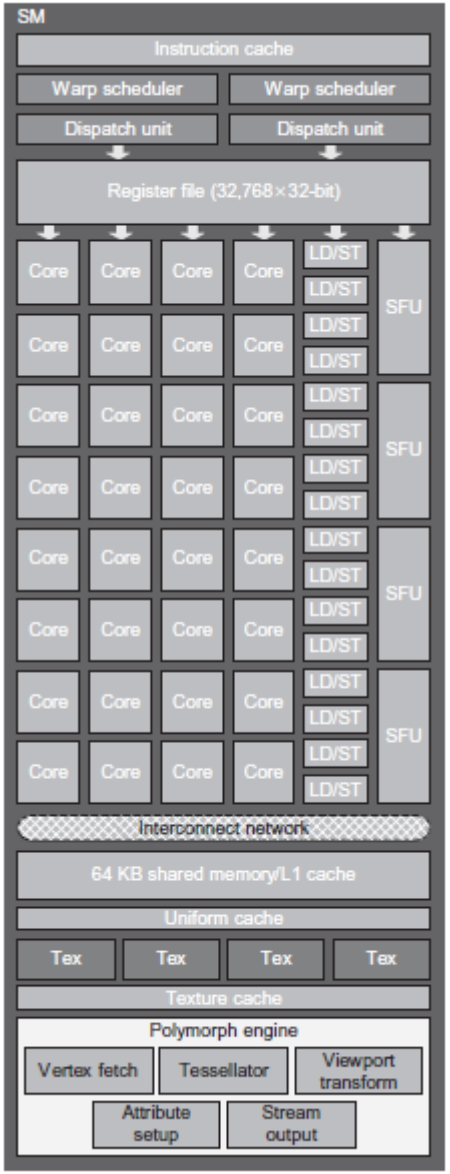

Ready Queue
Executing
Suspended
Warp 0 (Theads 0 to 31)
Warp 1 (Theads 32 to 63)
Warp 2 (Theads 64 to 95)
Warp 3 (Theads 96 to 127)
Memory Request Pending
Address 0 to 31
Address 32 to 63
Address 64 to 95
Address 96 to 127
Scheduling Cycle 8


Ready Queue
Warp 1 (Theads 32 to 63)
Executing
Warp 0 (Theads 0 to 31)
Suspended
Warp 2 (Theads 64 to 95)
Warp 3 (Theads 96 to 127)
Memory Request Pending
Address 64 to 95
Address 96 to 127
Scheduling Cycle 9

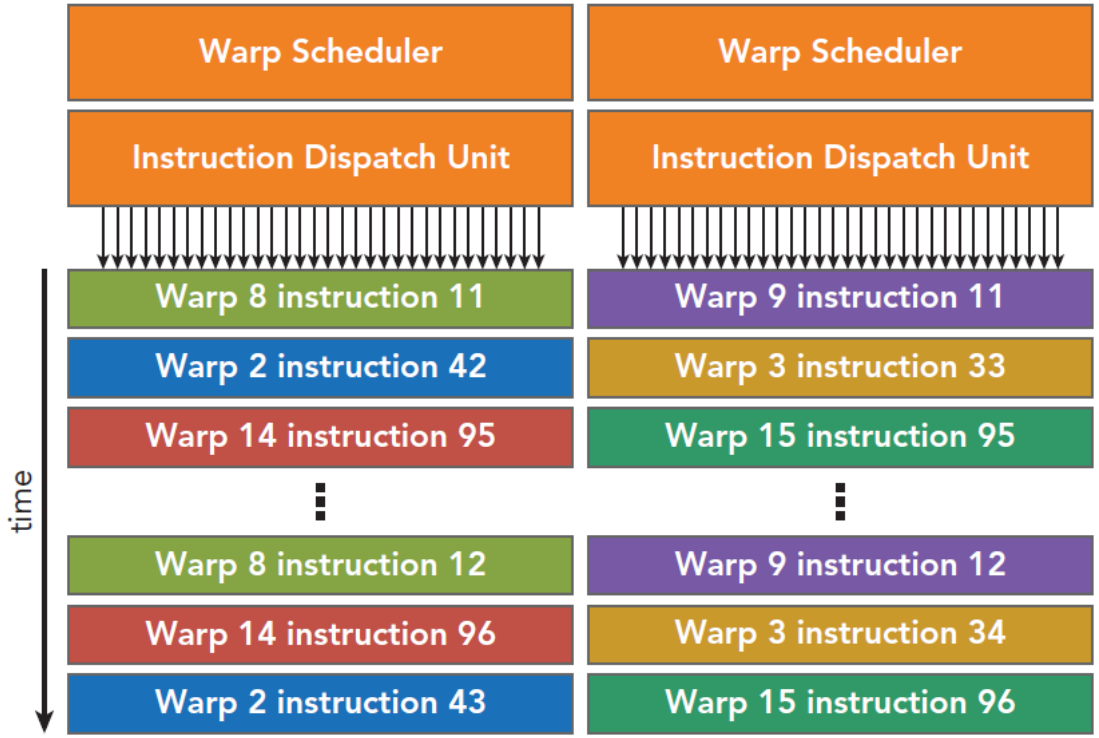
Fast Switching Between Warps

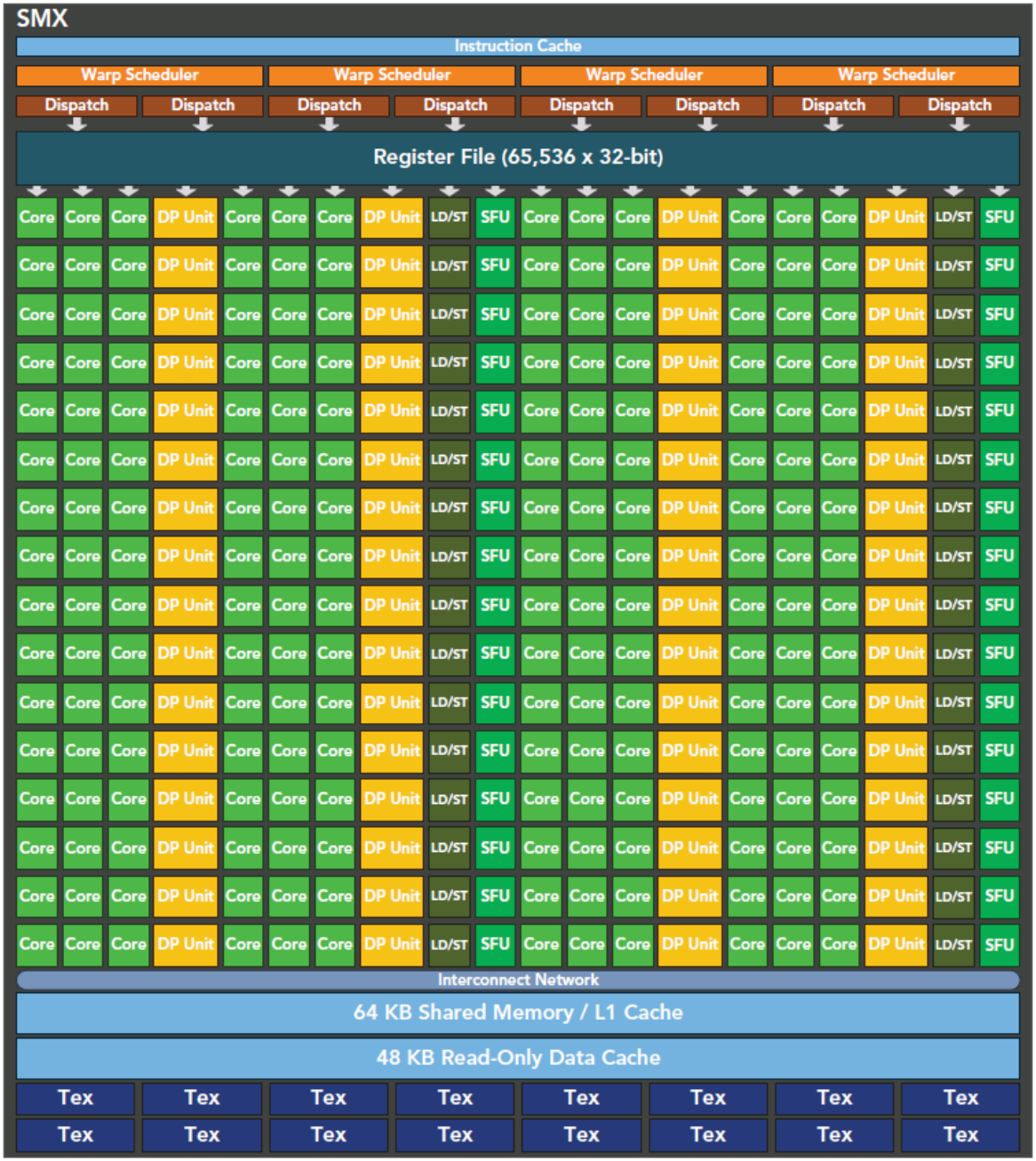
- SM implements zero-overhead warp scheduling
  - At any time, only a few warps are executed by SM
  - Warps whose next instruction has its operands ready for consumption are eligible for execution
  - Eligible Warps are selected for execution on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected

Warp Scheduling on Fermi GPUs



SM in Fermi architecture

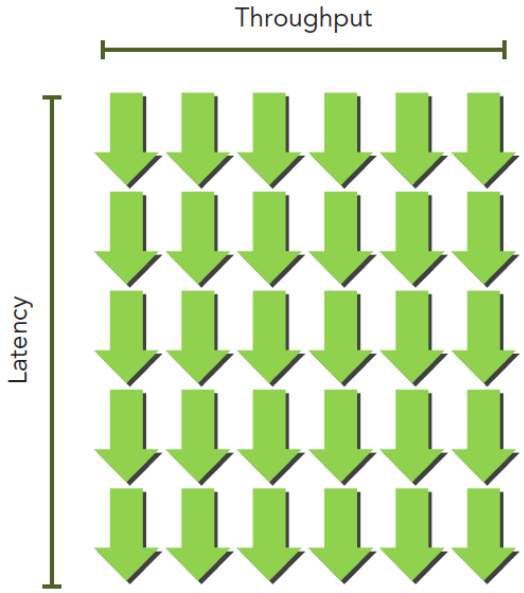




SM (SMX) in Kepler architecture

Little's Law

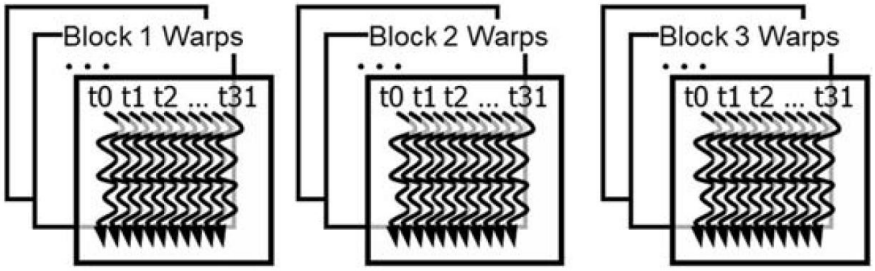
Number of warps = latency x warp throughput

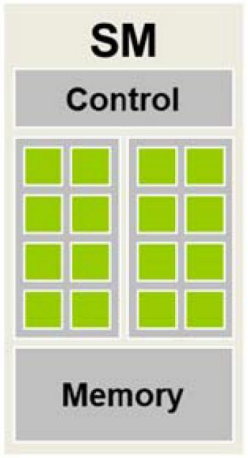


Arithmetical latency

GPU MODEL	INSTRUCTION LATENCY (CYCLES)
Fermi	20
Kepler	20

Occupancy





If 6 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?

- Each Block is divided into 256/32 = 8 Warps
- There are 8 * 6 = 48 Warps

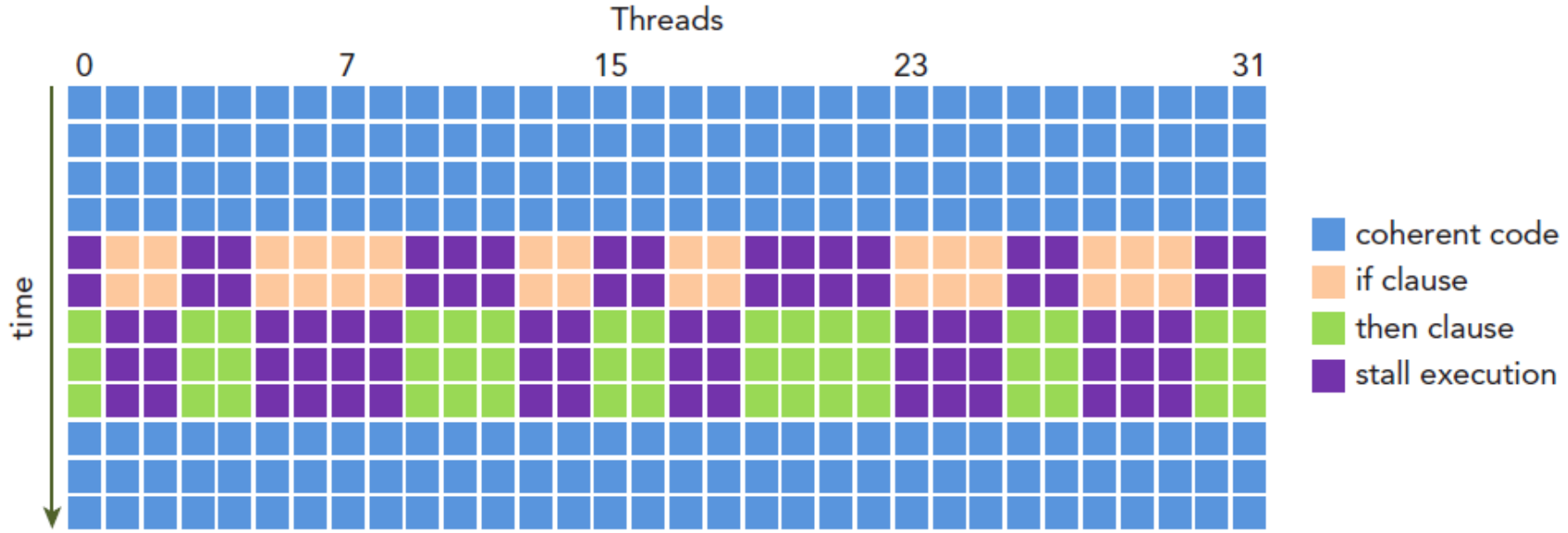
```
Occupancy =
  number of resident thread warps
    / max. number of resident warps
```

Warp Divergence

Conditionals (e.g., *if* statements) decrease performance of a warp by a factor of N (for N-nested *if*), as SM executes each branch serially.

```
__global__ some_func(void)
{
  if (some_condition)
  {
    action_a();
  }
  else
  {
    action_b();
  }
}
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+


Threads
0
7
15
23
31
time
coherent code
if clause
then clause
stall execution

Short Conditional Statements

Exception: really short conditionals (7 or 4 instructions) are executed using the *predication*

Example:

```
if (x<0.0) z = x-2.0;
else z = sqrt(x);
```

```
p = (x<0.0);        // logical predicate
  p: z = x-2.0;     // True predicated instruction
 !p: z = sqrt(x);   // False predicated instruction
```

If the condition evaluates the same for all threads in a warp, there is no performance reduction

Compiler determines this by either inserting warp voting functions, or, sometimes it is possible to do without them.

Example I:

```
//The variable case has the same value across all threads
if (case==1)
z = x*x;
else
z = x+2.3;
```

Example II:

```
if ((thread_idx % 32) < 16)
{
  action_a();
}
else
{
  action_b();
}
```

```
__global__ void mathKernel1(float *c) {
   int tid = blockIdx.x * blockDim.x + threadIdx.x;
   float a, b;
   a = b = 0.0f;

   if (tid % 2 == 0) {
      a = 100.0f;

   } else {
      b = 200.0f;
   }
   c[tid] = a + b;
}
```

```
__global__ void mathKernel2(void) {
   int tid = blockIdx.x * blockDim.x + threadIdx.x;
   float a, b;
   a = b = 0.0f;

   if ((tid / warpSize) % 2 == 0) {
      a = 100.0f;
   } else {
      b = 200.0f;
   }
   c[tid] = a + b;
}
```

Drawback of TLP (Thread Level Parallelism)

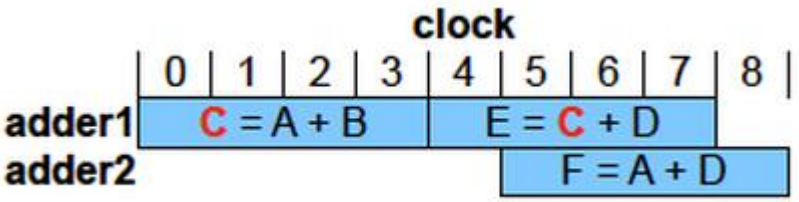
- TLP is wasteful: for(int i=0; i<n; i++) …
  - Uses 512 registers when threads per block are 512
  - Only 64 when threads per block are 64

- Registers are valuable
  - Only memory fast enough for peak performance
  - (Volkov 2010) Register 8 TB/s; shared mem 1.3 TB/s: Global mem 140 GB/s

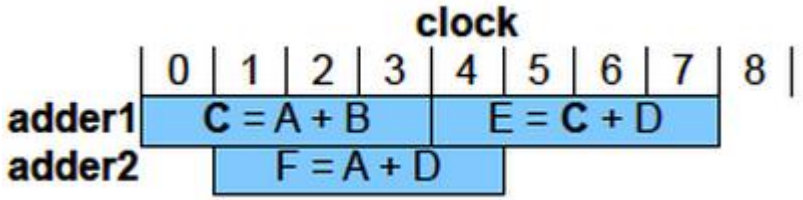
GF100	**20 at 100% occupancy**	**63 at 33% occupancy**	**3x more registers per thread**
GF200	16 at 100% occupancy	≈ 128 at 12.5% occupancy	8x more registers per thread

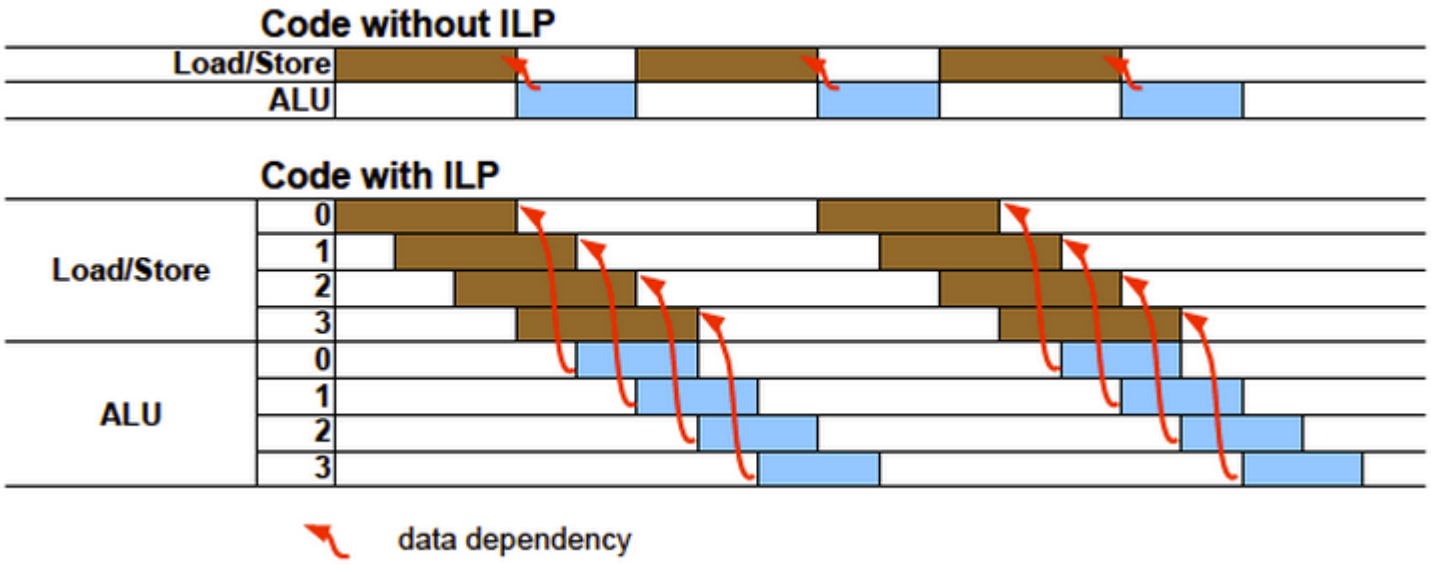
Instruction Level Parallelism (ILP)

```
C = A + B
E = C + D
F = A + D
```




```
C = A + B
F = A + D
E = C + D
```



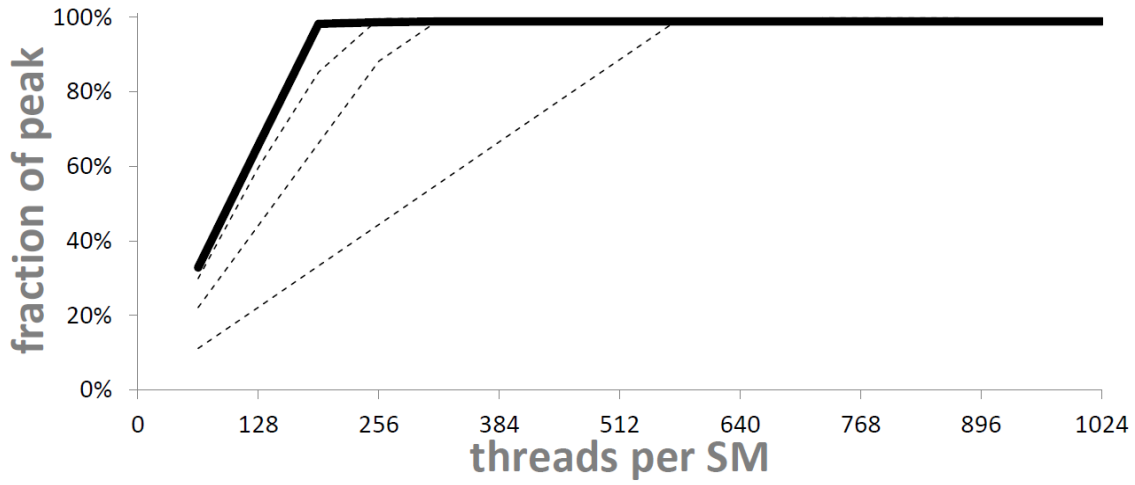


Reading source on the ILP: **Understanding Latency Hiding on GPUs,**
by Vasily Volkov (Ph.D. thesis, UC Berkeley)

Advantage of ILP (Instruction Level Parallelism)

```
#ifdef ILP4
#pragma unroll 16
  for(int i=0; i < NUM_ITERATIONS; i++) {
    a = a * b + c;     d = d * b + c;    e = e * b + c;     f = f * b + c;
  }
#else
#pragma unroll 16
  for(int i=0; i < NUM_ITERATIONS; i++) {
    a = a * b + c;
  }
#endif
```

Four operations per thread!



ILP=4: need 192 threads to get 100% utilization

Homework Assignment #3 (4 points), due on 5.11.26

RUB

1) Take the matrix addition code from the previous homework and use 1000 x 1000 matrices. Change the execution configuration in each dimension with an increment of 2 and, utilizing device query API functions for your particular GPU, determine occupancy and execution time and provide corresponding 2D matrices (or, better yet, plot them if you can). Let your code automatically determine the execution configuration yielding the best performance (i.e., minimum execution time).
2) Demonstrate that warp control divergence leads to a performance deterioration. For example, one can take the previously discussed 1D vector addition example with a large enough vector and use hints given on the next page and compare execution times for different n (Note that one has to ensure the same total number of operations in all versions). Using instead some other example of your own is fine, too.

Follow the usual homework submission guidelines (see Lecture 2)!!!

```
#define Nmax 32
...

in the kernel:

ind = threadIdx.x + blockIdx.x*blockDim.x;

for (i=0; i<n; i++)
  if (threadIdx.x%32>=int(32*i/float(n)) && threadIdx.x%32<int(32*(i+1)/float(n)) )
    {c[ind] += i;}
for (i=n; i<Nmax; i++)
  if (threadIdx.x%32<int(32*n/float(n)) && threadIdx.x%32<int(64*n/float(n)) )
    {c[ind] += i;}

...

in the main:

for (n=1; n<Nmax; n++)
  vecAdd <<< Nblocks,128 >>> (A_d, B_d, C_d, Nelem, n);
```